

Cornfigurator: Automated Planning for Any-to-Any Multimodal Model Serving

Jeff J. Ma^{1,*} Jae-Won Chung^{1,*} Jisang Ahn¹ Yizhuo Liang² Runyu Lu¹
Akshay Jajoo³ Myungjin Lee³ Mosharaf Chowdhury¹

¹University of Michigan ²University of Southern California ³Cisco Research

Abstract

Any-to-Any models are an emerging class of multimodal models that accept combinations of text and multimodal data as input and generate them as output, introducing heterogeneous computation paths and component scaling characteristics. There are existing mechanisms for deploying Any-to-Any models—or special cases of them—for inference serving, but they either require manual effort and expertise to tune, or do not generalize to generic Any-to-Any models.

We present Cornfigurator, the first deployment planner for generic Any-to-Any model inference serving. The goal of Cornfigurator is to maximize the overall *goodput* of serving the model, defined as the throughput of requests meeting their latency targets. To do so, based on model and workload characteristics, Cornfigurator explores the full spectrum of deployment strategies, from colocation to disaggregation and mixing different strategies. Cornfigurator performs coarse-to-fine statistical evaluation to efficiently navigate the large space of candidate plans. Plans generated by Cornfigurator either match or deliver $1.12\times$ – $6.32\times$ higher goodput compared to existing systems and expert-tuned deployment plans. Cornfigurator is open-source available on Github¹.

1 Introduction

Going beyond text-only Large Language Models (LLMs), we see a rapid proliferation of multimodal models that process and generate not just text, but also images, video, and audio. These *Any-to-Any* models, with over 11,000 variants on Hugging Face as of March 2026 [14], can (1) understand multimodal inputs and/or (2) generate multimodal outputs alongside text. For instance, Multimodal LLMs (MLLMs) like Qwen VL [7, 27] and InternVL [46] process multimodal inputs and generate text (Figure 1a); Qwen Image [26, 34] and GLM Image [41] produce images using diffusion from text embedded by an LLM; LTX-2 [12] generates video and audio; DeepSeek Janus [8, 35] understands and generates both text and images; and Qwen Omni [28, 37, 38] accepts combinations of text, image, video, and audio as input and generates text and audio (Figure 1b). In essence, text-only LLMs or diffusion models that generate images and videos are *special cases* of Any-to-Any multimodal models.

The computations of Any-to-Any (A2A) models are defined as a *graph of heterogeneous components* that handle different modalities: multimodal encoders, autoregressive

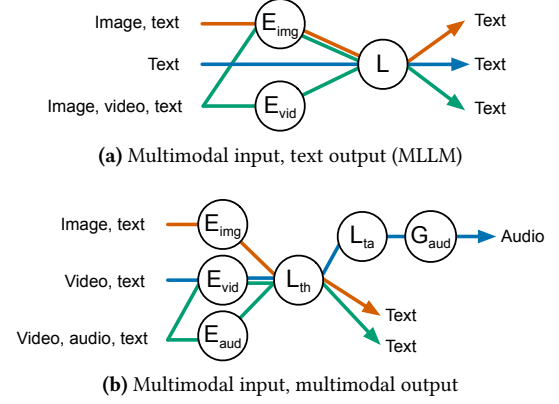


Figure 1. Requests invoking (a) a multimodal input model (InternVL [46]) and (b) a multimodal input and output model (Qwen Omni [28, 37, 38]). Different requests invoke different components of the model in different paths. *E* stands for Encoder, *L* for LLM, and *G* for Generator. *L_{th}* and *L_{ta}* stand for thinker and talker LLMs, respectively.

components like LLMs, and multimodal generators. The execution of A2A models is distinguished from that of traditional special cases by two new types of heterogeneity (§2.1). First, different *request types* traverse different computation paths through the graph. Figure 1 illustrates this with MLLMs and Qwen Omni as examples. Particularly, in Qwen Omni (Figure 1b), image, video, and audio encoders feed embeddings into a *thinker LLM* for text generation; then, if audio output was requested by the user, the thinker’s output is further passed to a *talker LLM* and then a vocoder, producing audio waveforms. Requests with different input and output modalities therefore invoke different subsets of components, leading to uneven per-component request rates. Second, different components have vastly different resource requirements and computational characteristics. As we show in Section 2.1, in Qwen 3 Omni, for instance, the thinker LLM achieves nearly $30\times$ higher request throughput than the talker LLM on A100. Without well-balanced resource allocation to heterogeneous components, the overall throughput of inference serving will be bottlenecked by the slowest component.

Existing works have handled component heterogeneity in large model serving by *disaggregating* computation into separate *executors* [2, 3, 25, 30]. On the one hand, systems like vLLM-Omni [3] and SGLang-Omni [2] provide mechanisms to disaggregate and run inference of generic A2A models, but require human experts to manually search for

*Equal contribution.

¹<https://github.com/cornserve-ai/cornfigurator>

a good deployment plan. On the other hand, systems like ModServe [25] and EPD [30] are designed for special cases of A2A models (e.g., MLLMs) and do not generalize to generic A2A models. Neither category of systems provides an automated planner for generic A2A models, and building such a planner is non-trivial; the best strategy can be complex and model-dependent, and finding the best strategy requires navigating a large and complex search space (§2.2).

To fill this gap, we build Cornfigurator, an automated deployment planner for generic A2A model inference serving (§3). Cornfigurator is designed to make the right colocation and disaggregation decisions for A2A models based on model and workload characteristics, instead of prescribing fixed strategies based on model architecture. The key insight of Cornfigurator is that we should reason about *each request type*, instead of lumping all requests together. This is because different request types invoke different amounts of computation and serve different purposes within an application, even though they are served by a *shared* model. For instance, an audio response and a text response may have different latency expectations. Thus, Cornfigurator’s optimization objective is to maximize the throughput of each request type constrained by each type’s own latency target, or in other words, to maximize the *goodput* of each request type.

To do so, Cornfigurator’s planning algorithm (§4) systematically explores colocation and disaggregation combinations for a given model, rather than prescribing a fixed strategy. The planner enumerates *logical subplans* (graph topologies) that may each *specialize* for different subsets of request types, merges subplans that share nodes into *compound subplans*, and composes them into *logical plans*. Logical plans are annotated with per-node executor configurations and routing probabilities to produce *physical plans*, which are concrete specifications of how to deploy and run the model. Then, the planner evaluates the per-request-type goodput of each physical plan through coarse-to-fine statistical evaluation: network flow for throughput ceiling, Monte Carlo sampling for latency, and a request-level simulator for accurate serving dynamics modeling, with pruning at each stage.

Cornfigurator is runtime-agnostic by design. We implement and evaluate it on top of Cornserve [1, 9], a distributed serving runtime for generic A2A models (§5). On a variety of recent A2A models including Qwen 3 Omni [38], Qwen 3 VL [27], InternVL 3 [46], and Qwen Image [34], Cornfigurator’s plans either match or deliver $1.12\times$ – $6.32\times$ higher goodput compared to plans used by existing systems (§6).

To summarize, our contributions are as follows:

- We identify a gap in automated deployment planning for generic Any-to-Any model serving.
- We present Cornfigurator, an automated planner for generic Any-to-Any models that maximizes goodput by reasoning about each request type and navigating deployment strategies and resource allocations.

Model	Input	Output
Qwen 2.5/3/3.5 Omni [28, 37, 38]	T, I, V, A	T, A
Qwen 2.5/3 VL [7, 27], InternVL 3 [46]	T, I, V	T
DeepSeek Janus [8, 35]	T, I	T, I
LTX-2 [12]	T, I	V, A
Qwen Image [34], GLM Image [41]	T	I

Table 1. Input and output modalities of recent Any-to-Any multimodal models. Modalities (Text, Image, Video, Audio) supported by a model can vary significantly.

Model	Image input	Video input	Audio input	Text in/out	Audio output	Image output
Qwen 3 Omni	5.43	2.93	21.43	2.15	0.12	
Qwen 2.5 Omni	15.64	1.28	34.04	1.09	0.28	
Qwen 3 VL	8.95	0.97		1.56		
Qwen 2.5 VL	12.04	0.89		1.63		
InternVL 3	1.13	0.74		0.59		
Qwen-Image				15.67		0.20

Table 2. Per-component throughput (requests/s) of various Any-to-Any models on A100-80GB GPU. Empty cells mean the model does not have that component.

- We evaluate Cornfigurator on a variety of recent Any-to-Any models and show that its plans match or deliver higher goodput compared to plans in existing systems.

2 Background and Motivation

We provide background on Any-to-Any multimodal models (§2.1), and discuss the deployment configuration space that motivates the need for an automated planner (§2.2).

2.1 Any-to-Any Multimodal Models

Recent models embrace multimodality at their core, naturally so because our world is full of multimodal information and interactions. This gave rise to a new class of models called *Any-to-Any* (A2A) multimodal models. As Figure 1 shows, A2A model computations form a graph of heterogeneous components. In essence, traditional text-only Large Language Models (LLMs) and Diffusion Transformer (DiT) models are *special cases* of A2A models where all requests traverse the same path through a linear pipeline of components.

When it comes to deployment and inference, the graph-based computation structure of A2A models introduces new sources of heterogeneity. Table 1 summarizes the input and output modalities supported by recent A2A models. Requests with different combinations of input and output modalities invoke different subgraphs—we call each combination with a unique subgraph a *request type*. An incoming inference workload contains a mix of request types, leading to each component experiencing a different request rate. Furthermore, different components have vastly different resource requirements and computational characteristics. Table 2 shows the per-component throughput of recent A2A models on an

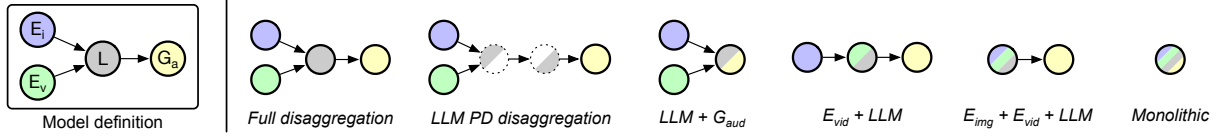


Figure 2. Graph-level deployment strategies for a four-component Any-to-Any model. The leftmost graph shows the model’s component graph definition: $(E_{img}, E_{vid}) \rightarrow LLM \rightarrow G_{aud}$. Graphs to the right show various example graph-level deployment strategies, which differ in how they group or split components into nodes (circles). Strategies shown are non-exhaustive.

A100-80GB GPU.² Component throughput can differ by orders of magnitude within the same model: in Qwen 3 Omni, the throughput difference between audio input and output components is nearly 200 $\times$. Together with request type heterogeneity, component heterogeneity leads to even more uneven per-component load.

2.2 Deployment Strategies and Configurations

The heterogeneity of A2A models motivates specialized deployment strategies—namely, grouping components into *nodes* (a colocated set of components; graph-level), and configuring each node’s execution (executor-level). These decisions are associated with complex tradeoffs and lack any silver bullet strategy optimal for all models and workloads.

Graph-level decisions. At the graph-level, the main decision is how to group components into graph nodes. Prior works focused on MLLMs, a special case of A2A models that consist of an LLM and one or more multimodal encoders. The simplest strategy deploys the entire model in a single *monolithic* executor (e.g., vLLM [18]), but this couples the execution and scaling of all components, making the slowest component the bottleneck for the entire model. *Disaggregated* strategies decouple the execution and scaling of different components by placing them on separate devices. For MLLMs, we may disaggregate just the encoders (e.g., ModServe [25]) or further disaggregate the LLM into separate Pre-fill and Decode phases (e.g., EPD disaggregation [30]). Going further, systems like vLLM-Omni [3] and SGLang-Omni [2] provide generalized component-wise and/or phase-wise disaggregation for A2A models. Figure 2 shows graph-level strategies for an example four-component A2A model.

Executor-level decisions. After we make graph-level decisions, we know which components are grouped together in an executor. Each executor exposes its own configuration knobs that control its throughput, latency, and GPU resource consumption. For instance, nearly all executors can be configured with a maximum *batch size* and a *parallelism degree* with different parallelism strategies available depending on the executor and component (e.g., tensor/expert parallelism for LLMs [18], sequence parallelism for DiTs [20]).

vLLM 4 $\times$ Monolithic (TP2, BS32)	4.39	5.62	4.68	3.39	29.8	23.2	27.6	38.4
vLLM 4 $\times$ Monolithic (TP2, BS64)	4.41	5.86	5.98	2.53	52.1	41.1	43.2	77.9
ModServe 2 $\times$ E+3 $\times$ L (TP2, BS32)	3.32	5.15	4.75	2.90	8.2	15.9	21.0	23.3
ModServe 2 $\times$ E+3 $\times$ L (TP2, BS64)	3.26	6.07	5.85	2.86	8.8	28.3	28.3	23.8
ModServe 4 $\times$ E+1 $\times$ L (TP4, BS32)	2.96	3.13	1.86	1.54	12.1	11.0	18.1	22.6
ModServe 4 $\times$ E+1 $\times$ L (TP4, BS64)	3.42	3.68	2.93	2.09	20.0	18.1	22.5	32.1
EPD 2 $\times$ E+1 $\times$ P+2 $\times$ D (TP2, BS32)	0.63	0.66	1.11	0.57	7.6	7.2	21.9	20.0
	W1	W2	W3	W4	W1	W2	W3	W4

(a) Throughput (req/s)

(b) p90 latency (s)

Figure 3. InternVL 3 [46] (a) throughput and (b) p90 latency under different deployment strategies and workloads³ on 8 $\times$ A100-80GB GPUs. Each row uses a graph-level strategy from prior work; numbers before E/L/P/D indicate executor instances. LLM (L/D) tensor parallel degree (TP) and batch size (BS) are shown in row labels; disaggregated encoders (E) and prefill (P) always use TP1 BS1 and TP2 BS1, respectively.

No silver bullet. Disaggregation, while proposed by many prior works, is not universally beneficial. Placing a component on its own GPU lets it scale independently, but it also consumes GPU resources that could otherwise be used by other components. For instance, disaggregating a multimodal encoder from the LLM allows independent scaling, but the GPUs allocated to the encoder can no longer store the KV cache for the LLM, which may reduce the LLM’s throughput. At the executor-level, increasing parallelism degree reduces per-request latency by distributing compute across more GPUs, but may hurt throughput (and eventually latency) due to communication overhead. Similarly, increasing batch size improves throughput until the GPU saturates, but it may hurt latency [19]. Navigating these tradeoffs and determining graph- and executor-level configuration knobs is highly dependent on the workload and model architecture, as shown by prior work on LLM serving [6, 45], DiT serving [20], and configuration tuning [23, 39]. Figure 3 quantifies this for InternVL 3 38B [46]. The heatmap shows the throughput and latency of different graph- and executor-level configurations under different workloads on 8 $\times$ A100-80GB GPUs. Even for this relatively simple single-encoder MLLM, there is no

²Requests are from ServeGen [36]. We use executor default configurations, except for LLMs larger than 30B for which we use tensor parallel degree 2.

³Workloads are specified as (input length, output length, image resolution, number of images per request): W1=(100, 100, 1920 $\times$ 1080, 1), W2=(1000, 100, 896 $\times$ 896, 1), W3=(100, 300, 896 $\times$ 896, 1), W4=(100, 300, 896 $\times$ 896, 2).

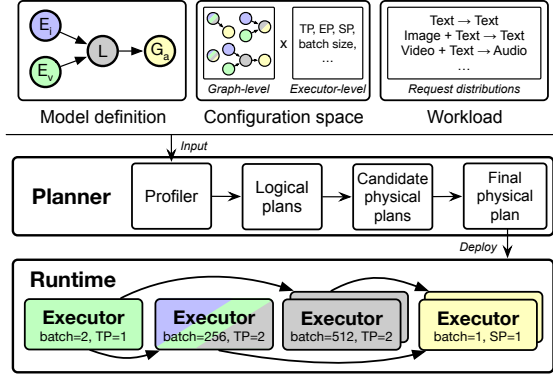


Figure 4. Planning and deployment with Cornfigurator. The planner takes as input the model definition, configuration space, and workload; the profiler benchmarks model components and the planner produces a physical plan. Cornfigurator assumes an existing serving runtime that deploys executors on GPUs and runs requests as planned.

silver bullet strategy; the best-throughput and best-latency configurations vary across workloads.

Automated planning. Navigating the vast configuration space of A2A model deployment (order of *millions*; §6.8) to find the best strategy is non-trivial, and existing works are limited. They either are designed for special cases of A2A models and prescribe fixed deployment strategies based on model architecture [25, 30], or only provide mechanisms to disaggregate and run specific A2A models [2, 3]. Building an efficient automated planner that can search for good deployment plans for generic A2A models is the goal of this paper.

3 Cornfigurator Overview

Cornfigurator is a deployment planner for Any-to-Any model inference serving. This section presents the planner’s location and role in the serving stack (§3.1) and the planning objective (§3.2). Section 4 describes the planning algorithm.

3.1 Planning and Deployment Architecture

Figure 4 shows how the planner fits into the serving stack. The planner supplies an existing *serving runtime* with a *physical plan* derived from the following inputs:

- **Model definition:** A directed acyclic graph whose nodes are model components and edges are data dependencies.
- **Configuration space:** The set of all executor types the runtime can instantiate, each specifying which model component(s) it handles, from which other executor types it can receive input, and available executor-level configurations. This determines the feasible graph-level and executor-level configurations (§2.2).
- **Workload:** A representative set of requests that follows the expected distribution of request types T , with per-type fractions π_t ($\sum_{t \in T} \pi_t = 1$).

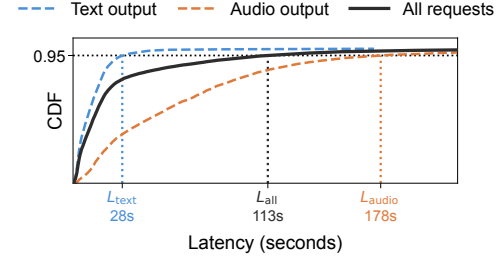


Figure 5. Latency CDFs of text-output requests (blue), audio-output requests (orange), and all requests together (black). Audio-output requests require more computation as they must go through the talker LLM and vocoder. A global latency constraint L_{all} binds only on the heavier audio-output requests, while per-type latency targets L_{text} and L_{audio} ensure each request type is individually constrained.

- **Parameters:** Number of homogeneous GPUs available N and latency targets L_t for each request type.

Given these inputs, the *profiler* (§5.1) benchmarks each model component under the target workload, recording the throughput and latency of each component on the target hardware. Using these profiles, the *planner* (§4) produces a set of *physical plans*, each specifying the nodes, the number of executors for each, their configurations, and routing probabilities when multiple paths are available for a request type. The final physical plan (§3.2) is deployed to the serving *runtime*, which instantiates executors on the GPUs and serves requests according to the plan.

3.2 Planning Objective

The planner’s goal is to maximize throughput subject to latency constraints. A common formulation is to constrain a single global statistic of request latency, but this can be problematic for A2A models. First, with input and/or output modality differences between request types, the application context within which the request’s response is used may differ, leading to different latency expectations per type. For instance, users may tolerate longer waits for videos than for images. Furthermore, in generic A2A models, requests of different types take different paths through the model, leading to widely different computation costs and latency distributions. Figure 5 illustrates such a case, where an example A2A model has two request types: text output (blue; lighter) and audio output (orange; heavier). When a single global latency target L_{all} is used, it binds only on the heaviest request type (audio), while the lighter type (text) faces no effective constraint; nothing stops the planner from freely degrading the latency of text requests.⁴ Therefore, Cornfigurator instead imposes latency constraints on *each request type independently*, ensuring that the latency of each

⁴Lighter types with less compute requirement are especially vulnerable. Since their latency is far below the global threshold, they can absorb substantial degradation before ever pressuring the global constraint.

type is constrained according to its own requirements and computation cost, and that the planner cannot arbitrarily degrade the latency of lighter request types.

Latency constraints. Cornfigurator allows user to set latency targets L_t per type in any way that reflects their needs. One natural default/starting point is to set L_t proportionally to the computation cost of type t , so that a request requiring twice the computation gets twice the latency budget. Appendix A proves that this choice factors out scale differences between types, making the constraint equally tight for all types when their latency distributions have similar shapes.

Maximizing goodput. Per-type latency constraints motivate a per-type metric that captures both throughput and latency compliance. We define the *goodput* of request type t for a physical plan as the throughput of type- t requests that meet their latency target. Goodput has desirable properties. A plan that violates latency targets gets penalized even if its raw throughput is high, and when a plan cannot handle incoming load, queue build-up increases latency and goodput tends to zero. For each candidate plan, the planner produces a vector of per-type goodput estimates. The planner's goal is to find the plan that maximizes overall goodput, summed across all request types.

4 Planning Algorithm

This section describes the planning algorithm. We first walk through the process at a high level (§4.1), then describe the two big stages: enumerating valid plans (§4.2), and evaluating them to find the best one (§4.3). Finally, we discuss how the planner can adapt to deployment scenario changes (§4.4).

4.1 Planning Overview

The goal of the planner is to explore diverse graph- and executor-level configurations and *mixtures* of them to find smoother tradeoffs that strike the right balance for the given model, workload, and GPU budget.

Executor-level configurations can only be decided after the *topology* of the graph is determined. As such, the planner begins at the graph-level. First, the planner enumerates *simple logical subplans* by exploring colocation and disaggregation decisions on the model definition graph, and then merges simple subplans that share nodes into *compound logical subplans* with internal parallelism and routing. These simple and compound logical subplans are building blocks, and each may specialize for different subsets of request types. Logical subplans are then composed into full *logical plans* with parallel paths that collectively cover all request types. These are then refined into *physical plans* with concrete GPU allocations, configurations, and routing probabilities (§4.2).

Each physical plan is then evaluated for per-request-type goodput (§4.3): a cheap statistical estimate prunes unpromising candidates, and a more accurate request-level simulator

Input: Model $G = (C, E)$, colocatable edges $E_c \subseteq E$
 Config space $\mathcal{K}_v$ for each possible node v
 Request types T , GPU budget N
 Subplan merge limit k_c , composition limit k_s
Output: Physical plans $\mathcal{P}$

► Simple subplans: colocation/disaggregation decisions

```

1  $\mathcal{L}_{\text{simple}} \leftarrow \emptyset$ 
2 for each valid subgraph  $G' = (C', E') \subseteq G$  do
3   for each  $m \in \{\text{KEEP}, \text{MERGE}\}^{|E' \cap E_c|}$  do
4      $L \leftarrow$  fully disaggregated plan for  $G'$ 
5     for each edge  $e = (c_i, c_j) \in E' \cap E_c$  do
6       if  $m_e = \text{MERGE}$  then Merge  $c_i, c_j$  into  $c_{ij}$ 
7     Add  $L$  to  $\mathcal{L}_{\text{simple}}$ 
8  $\mathcal{L} \leftarrow \mathcal{L}_{\text{simple}}$ 
9 for each subset  $\{L_1, \dots, L_j\} \subseteq \mathcal{L}_0, 2 \leq j \leq k_c$  do
10   if share nodes then Add overlay( $L_1, \dots, L_j$ ) to  $\mathcal{L}$ 
11  $\mathcal{S} \leftarrow \emptyset$ 
12 for each multiset  $S$  over  $\mathcal{L}$  where  $|S| \leq k_s$  do
13   if covers all request types then Add  $S$  to  $\mathcal{S}$ 
14  $\mathcal{P} \leftarrow \emptyset$ 
15 for each  $S \in \mathcal{S}$  do
16   for each valid executor partition do
17     for each routing probability assignment do
18       Add  $(S, \text{executors}, \text{routing})$  to  $\mathcal{P}$ 
19 return  $\mathcal{P}$ 
```

Algorithm 1: Plan enumeration.

evaluates the survivors. Finally, the physical plan that maximizes overall goodput is selected for deployment.

4.2 Plan Enumeration

Algorithm 1 summarizes the plan enumeration stage, and Figure 6 provides a running example. Given inputs (Figure 6a; §3.1), the planner enumerates candidates from logical subplans, to logical plans, and finally to physical plans. Logical (sub)plans define empty *variables* for GPU allocations, executor configurations, and routing probabilities. Physical plans *fill in* all such variables, producing complete specifications that can be deployed and served by a runtime.

Simple logical subplans. A *logical subplan* is a directed acyclic graph, where each node handles one or more model

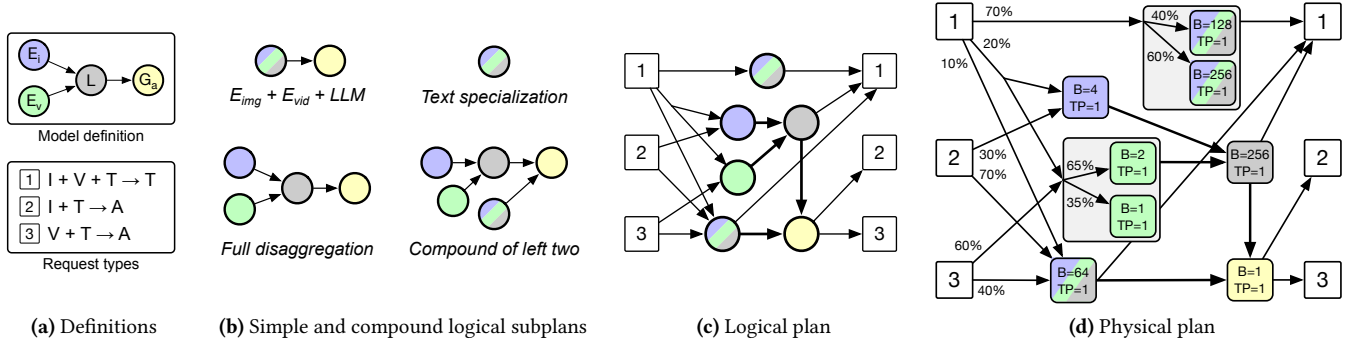


Figure 6. Running example for the plan enumeration phase. (a) Definition of a four-component model $((E_{\text{img}}, E_{\text{vid}}) \rightarrow L \rightarrow G_{\text{aud}})$ with three request types ($[1] I+V+T \rightarrow T$, $[2] I+T \rightarrow A$, and $[3] V+T \rightarrow A$). (b) Example subplans (not exhaustive). Simple: $E_{\text{img}}+E_{\text{vid}}+LLM$ with disaggregated G_{aud} , full disaggregation, and $E_{\text{img}}+E_{\text{vid}}+LLM$ without G_{aud} (specialized to type $[1]$). Compound: merging the left two simple subplans sharing G_{aud} . (c) An example logical plan composing the compound subplan and the text specialization simple subplan into a supergraph. Audio-output types ($[2]$ and $[3]$) must go to the compound subplan (only path with G_{aud}); type $[1]$ can go to either. Arrows that send text data to LLM executors are omitted for simplicity. (d) An example physical plan based on the logical plan in (c), allocating 8 GPUs, with per-node executors and their configurations, and per-type routing probabilities at the compound subplan entrypoint and across the two subplans.

components. The planner first constructs *simple* subplans by selecting a subgraph of the model definition that (1) covers at least one request type (not necessarily all types), and (2) has no unused components (i.e., every component in the subgraph is used by at least one covered request type). Each pair of adjacent components connected by a *colocatable* edge, as determined by the runtime, has a choice of whether to KEEP as separate nodes or MERGE into a single colocated node. The Cartesian product of these binary decisions (Algorithm 1, line 3) across all colocatable edges produces all valid node topologies for that subgraph—from fully disaggregated (all KEEP) to monolithic (all MERGE).

Compound logical subplans. The planner then generates *compound* subplans (Algorithm 1, line 9) by merging up to k_c simple subplans (default $k_c = 2$) that share one or more nodes. Requests still follow one subplan end-to-end, and per-type routing probabilities at the entrypoint of the compound subplan determine how traffic is split between merged subplans. Figure 6b shows example simple and compound subplans.

Constructing logical plans. A single logical subplan covers a subset of request types and occupies a portion of the GPU budget. This design is intentional; it allows the planner to include subplans that are specialized to different request types. For instance, one subplan may be optimized for text-output requests, while another handles audio-output requests (Figure 6b). By having subplans that specialize for different subsets of request types, the planner can mix and match them as appropriate to produce smoother tradeoffs. Given all subplans, the planner composes up to k_s subplans (default $k_s = 2$) into a *logical plan* (Algorithm 1, line 12): a supergraph with common entry and exit nodes, where each

subplan forms an alternative parallel path. A logical plan covers every request type with at least one path (Figure 6c).

Constructing physical plans. A physical plan (Figure 6d) is a logical plan annotated with concrete GPU allocations, executors and their configurations, and routing probabilities—it can be deployed to and run by a runtime.

First, for each node, the planner decides how many executors to run and what configuration (e.g., parallelism degree,⁵ batch size) each executor uses. This enumeration is analogous to the coin change problem: each executor type is a “coin” whose denomination is its GPU cost, and allocating executors to nodes is equivalent to finding all ways to spend between 1 to N GPUs. Unlike standard coin change, however, coins are not fungible even when they have the same denomination: two executors that cost the same number of GPUs but differ in other configurations (e.g., batch size) are considered different. The planner enumerates all physical plans (Algorithm 1, line 16) satisfying:

$$\begin{aligned} \sum_{v \in S} \sum_{k \in \mathcal{K}_v} a_{v,k} n_{v,k} &\leq N && \text{(Within GPU budget)} \\ \sum_{k \in \mathcal{K}_v} n_{v,k} &\geq 1 \quad \forall v \in S && (\geq 1 \text{ executor per node}) \end{aligned} \quad (1)$$

where v is a node in logical plan S , $\mathcal{K}_v$ is its set of feasible configurations (determined by Configuration space in §3.1), $a_{v,k}$ is the GPU cost of node v using configuration k , and $n_{v,k}$ is the number of executors with config k assigned to node v .

⁵The concrete parallelism strategy depends on the component type: tensor parallelism (TP) for LLMs and Vision Transformers, expert parallelism (EP) for MoE models, and sequence parallelism (SP) for Diffusion Transformers.

Input: Physical plans $\mathcal{P}$
 Executor throughput and latency profiles
 Workload $\mathcal{W}$ with request type fractions $\{\pi_t\}$
 Latency targets $\{L_t\}$ and headroom α

Output: Selected physical plan d^*

► **Phase 1: Flow-based throughput estimation**

- 1 Sort $\mathcal{P}$ by increasing executor capacity per node
- 2 **for** each $d \in \mathcal{P}$ **do**
- 3 Find bottleneck node in d
- 4 $R_d \leftarrow$ aggregate rate that saturates the bottleneck
- 5 Drop plans with redundant capacity from $\mathcal{P}$
- **Phase 2: Latency estimation with Monte Carlo**
- 6 **for** each $d \in \mathcal{P}$ **do**
- 7 Sample K requests from $\mathcal{W}$
- 8 **for** each request type t **do**
- 9 Route type- t requests through d
- 10 $F_{t,d} \leftarrow$ CDF of accumulated latencies
- 11 ► $\alpha R_d \pi_t$ incoming rate, $F_{t,d}(L_t)$ met latency target
- 11 $G_{t,d} \leftarrow \alpha R_d \pi_t \cdot F_{t,d}(L_t)$
- 12 Drop Pareto-suboptimal plans from $\mathcal{P}$
- **Phase 3: Request-level simulation**
- 13 **for** each $d \in \mathcal{P}$ **do**
- 14 $\{R_{t,d}, F_{t,d}\}_{t \in T} \leftarrow$ simulate d at rate $\alpha \cdot R_d$
- 15 $G_{t,d} \leftarrow R_{t,d} \cdot F_{t,d}(L_t)$ for each t
- 16 Drop Pareto-suboptimal plans from $\mathcal{P}$
- **Select: Default Policy is sum of goodput across types**
- 17 $d^* \leftarrow \arg \max_d \text{Policy}(G_{1,d}, \dots, G_{|T|,d})$
- 18 **return** d^*

Algorithm 2: Three-phase evaluation and selection.

Finally, the planner assigns per-type routing probabilities (Algorithm 1, line 17) at two levels: between executors in the same node and between alternative parallel paths in the logical plan. At each level, routing probabilities for each request type sum to 1. By default, routing probabilities are discretized with a step size of 0.1 (10%), but finer granularities are supported at the cost of more physical plans to evaluate (§6.7). The final result is a list of physical plans, each a complete specification for the runtime for deployment and execution.

Time complexity. Enumeration is exponential in the number of model components and colocatable edges, but since these are in practice at most tens (Qwen Omni has 6 components), enumeration time is manageable (§6.8). The maximum number of subplans to merge (k_c) and to compose (k_s) bound the explosion—we vary these in Section 6.7.

4.3 Plan Evaluation and Selection

Plan enumeration produces a large set of candidate physical plans. The planner evaluates them in a coarse-to-fine pipeline

(Algorithm 2). For all enumerated physical plans, a cheap network flow-based estimate provides accurate aggregate throughput estimates, and Monte Carlo sampling adds per-type latency estimates. Pruning rules discard unpromising plans after each phase, and survivors are evaluated with a more expensive request-level simulator that refines the accuracy of per-type goodput estimates to select the final plan. The pruning rules are exact; they only drop plans that are guaranteed to be redundant or suboptimal.

Network flow-based throughput estimation. Offline profiling provides the maximum throughput of each executor for each request type, and each node’s throughput (i.e., capacity) is the sum of its executors’ throughput. Given the workload’s per-type fractions π_t , each type contributes $R \cdot \pi_t$ to a node’s load, split across executors according to routing probabilities. Prior works have used (max) flow-based throughput analysis for a *single* request type [21, 45]. However, for A2A models, multiple request types flow through the graph simultaneously and share node capacity; at each node, the demands of all types passing through it are summed and compared against the node’s capacity. The node whose aggregate demand first reaches its capacity is the bottleneck, and the aggregate request rate R_d at which this occurs is plan d ’s maximum throughput (Algorithm 2, line 3). Plans that configure a node with more capacity than the node’s aggregate demand are redundant, as they won’t improve throughput or latency; all such plans are pruned (Algorithm 2, line 5).

R_d is an ideal upper bound; system overheads and request bursts prevent it from being achieved in practice. The planner therefore scales it down to $\alpha \cdot R_d$ (we find that the planner is robust to the setting of α ; §6.7), which is used as the incoming aggregate request rate for subsequent phases.

Latency estimation with Monte Carlo. The planner estimates request latency by Monte Carlo simulation (Algorithm 2, line 9): it randomly samples K requests from the Workload, routes each through the plan, and accumulates per-executor processing latencies (sampled from profiling results) to produce per-type latency CDFs. This is cheap because each request’s latency is independent of others (i.e., queuing not modeled), but still provides a reasonable estimate because for well-balanced plans, queuing delay is bounded and latency is dominated by processing time. After latency estimation, plans whose per-type goodput vectors are Pareto-dominated are pruned (Algorithm 2, line 12).

Request-level simulation. Earlier phases reasoned about aggregate flow and independent requests to provide good estimates for *aggregate* goodput, but they are less precise for each request type because they do not precisely model queuing dynamics and inter-type interactions at shared nodes. Candidates that survived earlier phases are therefore evaluated with a *request-level simulator* (Algorithm 2, line 14) that models the full request processing pipeline, refining

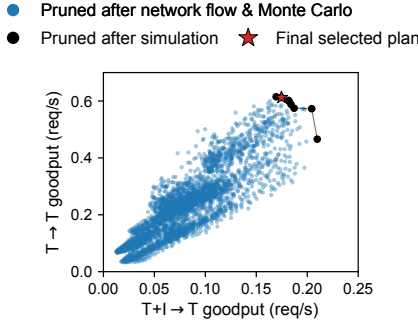


Figure 7. Illustration of how physical plan evaluation, pruning, and selection work. Each dot represents a physical plan, and the two axes are goodput for two request types. Plans are progressively pruned after each phase (network flow, Monte Carlo, and simulation), reflected by the dots’ color.

accuracy to the individual request type-level. The simulator runs the workload at rate $\alpha \cdot R_d$ through a physical plan and produces (1) a throughput estimate and (2) a latency CDF for each request type, from which goodput estimates per request type can be derived. Plans that are Pareto-dominated on their goodput vector are pruned, and the final plan is selected (Algorithm 2, line 17) according to a policy, which maximizes aggregate goodput by default. Section 6.5 evaluates the accuracy of the pipeline.

Figure 7 visualizes the goodput of physical plans of a model with two request types, with colors showing at which phase the plan was pruned. The plans pruned by network flow *overlap* in goodput values with their non-redundant counterparts as they have the same bottleneck node, so it reduces the number of candidate plans significantly but not the visible point cloud. Monte Carlo and request-level simulation eliminates Pareto-suboptimal plans.

Time complexity. Network flow and Monte Carlo are both linear in the number of candidate physical plans, and cheap per plan; the expensive request-level simulation runs only on the small set of survivors after pruning. Section 6.8 breaks down the number of plans and time spent in each phase.

4.4 Adapting to Changes

Three types of changes can occur in a deployment scenario, each requiring progressively more work to adapt.

GPU budget changes. If the GPU budget N changes but the workload distribution and model remain the same, the planner can simply re-run with the new N . Profiling results remain valid, and the plan enumeration and evaluation phases are fast enough to re-run semi-online (§6.8).

Workload changes. If request types fractions π_t change, profiling results can be reused because the profiler records per-request latency measurements. Changing the request type distribution is a matter of re-weighting existing samples. The planner re-runs enumeration and evaluation with the

updated workload fractions. On the other hand, if new request types are added, or when the characteristics of existing request types change (e.g., significantly different sequence lengths), the new/updated type needs to be re-profiled. We show this in Section 6.6.

Model or hardware changes. If the model architecture changes (e.g., a new component is added) or the hardware changes (e.g., migrating to a different GPU type), profiling must be re-run from scratch, as per-component throughput and latency characteristics are no longer valid. This is the most expensive adaptation, but it would also be the least frequent in practice.

5 Implementation

Cornfigurator is designed to be runtime-agnostic; it works with runtimes ranging from those supporting only a single executor type to fully generic A2A serving runtimes. As a proof of concept, we implemented Cornfigurator on top of Cornserve [1, 9], a distributed A2A model serving platform.

5.1 Profiler

The profiler benchmarks each supported executor under the workload to collect the throughput and latency data that the planner uses. The profiler deploys executors on the target hardware via the runtime and sends requests, sweeping over configuration knobs (e.g., batch sizes, parallelism degrees). Profiling is performed at a saturating request rate and measured only during the *steady state* window of the engine, excluding ramp up and down periods. In order to extract pure processing time, queuing delay is subtracted from end-to-end latency measurements using runtime traces.

5.2 Planner

The planner is implemented in about 5K lines of Rust. The plan evaluation phases (§4.3) implement the following extra optimizations to improve accuracy. The accuracy of Cornfigurator’s planner is evaluated in Section 6.5.

Occupancy-aware latency scaling. Flow-based estimation finds the bottleneck node in the plan that bounds the whole plan’s throughput. All other nodes are non-bottlenecks, and likely operate at a batch size smaller than its *configured* batch size, which lead to lower latency than what the profiler measured at the configured batch size. Therefore, during Monte Carlo simulation, the planner scales latency numbers for non-bottleneck nodes to reflect their effective batch size.

Modeling CPU-GPU overlap. Multimodal encoders perform CPU-GPU pipelining: the CPU preprocesses input data (e.g., image decoding) while the GPU runs the encoder model. When a request arrives at an idle multimodal encoder executor, it experiences the full CPU + GPU time. However, when the encoder is already busy, CPU preprocessing happens while the request is in the queue, and processing time

reduces to GPU time alone. This difference is significant especially when batch size is small. Thus, the simulator checks whether the executor is idle or busy and applies the corresponding processing time (CPU + GPU, or GPU alone).

Accounting for communication overhead. Intermediate tensors must be transferred between disaggregated executors. In our setup, the median transfer latency across representative tensor sizes and traffic rates is approximately 10 ms. While this transfer latency is not the bottleneck as computation dominates, the planner can account for it by adding this median delay at each disaggregated edge in the plan.

6 Evaluation

We plan the deployment of recent A2A models with diverse architectures, modalities, and request types with Cornfigurator and compare with state-of-the-art baselines. We find:

- Cornfigurator produces high-quality deployment plans for generic A2A models that achieve $1.24\times$ – $2.73\times$ higher goodput than baselines (§6.2).
- Cornfigurator produces plans that match or deliver $1.12\times$ – $6.32\times$ higher goodput over baselines for special case A2A models: multimodal input (§6.3) and output (§6.4) models.
- Cornfigurator’s planner estimates goodput with high accuracy (§6.5), supports adapting to workload drift (§6.6), is robust to parameter choices (§6.7), and produces plans with reasonable overhead (§6.8).

6.1 Common Evaluation Setup

Different A2A models have different input and output modalities, and thus different representative workloads and baselines. We describe common setup across all evaluations here, and architecture-specific setup in respective subsections.

Testbed. All experiments are run on two AWS p4de.24xlarge instances, each with $8\times$ NVIDIA A100-80GB GPUs connected via NVSwitch and 400 Gbps cross-node bandwidth.

Baselines. Baselines include plans generated by experts/developers of vLLM [18], vLLM-Omni [3], ModServe [25], and EPD [30].⁶ Cornfigurator is a strict superset of all baselines, so the plans that come with each baseline system can be expressed as a Cornfigurator plan. We deploy and evaluate all plans on Cornserve [1, 9], a generic A2A serving platform that natively supports Cornfigurator’s plan space to remove performance differences due to implementation differences [5].

Metric. Our primary performance metric is goodput (§3.2): the throughput of requests that meet their per-type latency targets. We compare the *maximum achievable* goodput of each plan, which is the goodput achieved by the highest request rate that the plan can support.

⁶SGLang-Omni [2] is not included as it is in early development stage, falling short on stability, model support, and deployment plans at the moment.

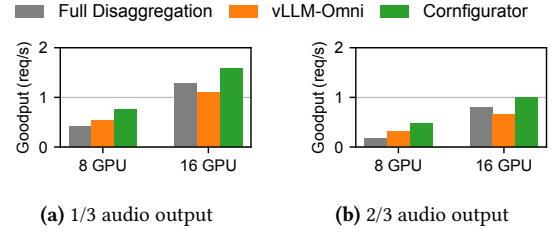


Figure 8. Maximum achievable goodput of Qwen 3 Omni under the ServeGen workload on 8 and 16 GPUs with (a) 1/3 and (b) 2/3 audio output requests.

Latency targets. We set the latency target of each request type (§3.2) in a manner that reflects the raw amount of computation requirements of that type, while still being achievable by a shared deployment plan that serves all types simultaneously. Unless mentioned otherwise, we identify the highest-throughput configuration for each request type served in isolation from offline profiling results and take its p25 latency as the target for that type. We show the sensitivity of goodput to the choice of percentile in Section 6.7.

6.2 Multimodal Input, Multimodal Output

Setup. We evaluate Qwen 3 Omni 30B [38] with the ServeGen [36] workload, which includes request timestamps. The workload has 8 request types: $\{T+I, T+I+V, T+I+A, T+I+V+A\} \rightarrow \{T, A\}$, where inputs always include text and image, and include video and/or audio input half of the time. We evaluate two audio output fractions (1/3 and 2/3) to study how the workload’s modality mix affects plans and goodput.

Baselines. We compare against two baselines. First is vLLM-Omni [3], which uses an expert-tuned but fixed deployment strategy. Second is Full Disaggregation, which is a restricted version of Cornfigurator; its graph-level configuration space is fixed to fully disaggregated (i.e., each model component running in a separate executor), but otherwise follows the full Cornfigurator planning pipeline to optimize executor-level configurations and GPU allocation.

Goodput comparison. Figure 8 compares goodput across systems on 8 and 16 GPUs under both audio output fractions. Cornfigurator achieves $1.24\times$ – $1.84\times$ higher goodput than baselines across both GPU budgets with 1/3 audio output (Figure 8a), and the gains widen to $1.25\times$ – $2.73\times$ with 2/3 audio (Figure 8b). As the audio generator becomes a heavier bottleneck, Cornfigurator’s ability to allocate resources proportionally to component demand becomes more valuable.

Scaling from 8 to 16 GPUs, both Cornfigurator and Full Disaggregation achieve super-linear scaling because the larger budget allows better component balance. However, Full Disaggregation still falls short of Cornfigurator because it disaggregates all components including those with less load, leading to underutilized GPU resource. In contrast, Cornfigurator automatically finds efficient colocation strategies.

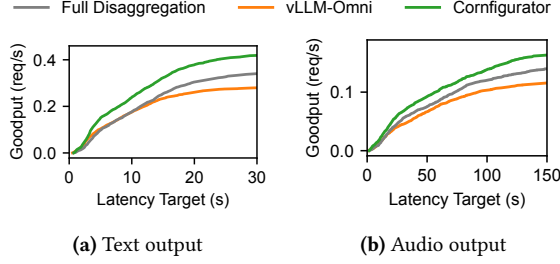


Figure 9. Goodput of requests in Qwen 3 Omni with T+I+V+A input with (a) text output and (b) audio output vs. latency targets under 1/3 audio workload on 16 GPUs.

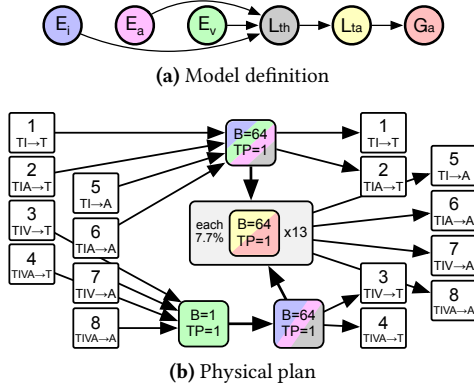


Figure 10. (a) Qwen 3 Omni definition graph and (b) Cornfigurator's deployment plan under 2/3 audio workload on 16 GPUs. The planner prepared a video-only subplan (with a disaggregated video encoder) for heavy video-input requests.

Per-type goodput vs. latency target. Figure 9 shows the goodput of two request types in Cornfigurator with 1/3 audio, 16 GPU deployment (remaining six types are in Appendix B). Cornfigurator's plans achieve higher goodput than baselines across the full range of latency targets, from tight to relaxed.

Physical plans. We examine the plans generated by Cornfigurator to understand its planning decisions. On the 1/3 audio workload (executor configs are similar and thus omitted):

- **8 GPUs:** $1 \times (E_{\text{img}}) + 2 \times (E_{\text{vid}} + E_{\text{aud}} + L_{\text{th}}) + 5 \times (L_{\text{ta}} + G_{\text{aud}})$
- **16 GPUs:** $1 \times (E_{\text{aud}}) + 4 \times (E_{\text{img}} + E_{\text{vid}} + L_{\text{th}}) + 11 \times (L_{\text{ta}} + G_{\text{aud}})$

Cornfigurator switches from disaggregating the image encoder on 8 GPUs to disaggregating the audio encoder on 16 GPUs, as the additional GPU budget allowed it to realize better balance. On the 2/3 audio workload, the 8 GPU plan is $1 \times (E_{\text{img}} + E_{\text{vid}} + E_{\text{aud}} + L_{\text{th}}) + 6 \times (L_{\text{ta}}) + 1 \times (G_{\text{aud}})$, reflecting that increased audio output load promoted talker LLM and vocoder disaggregation. With 16 GPUs (Figure 10), Cornfigurator has more GPUs to express higher degrees of specialization: the plan originates from a compound logical subplan (§4.2) that merges two simple subplans sharing $13 \times (L_{\text{ta}} + G_{\text{aud}})$. One

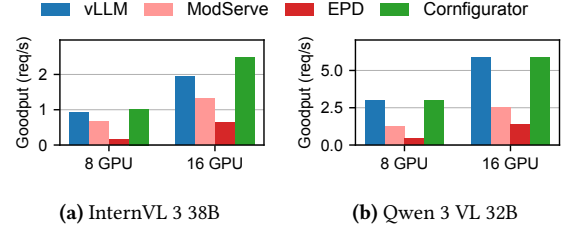


Figure 11. Maximum achievable goodput of two MLLMs under the ServeGen workload on 8 and 16 GPUs.

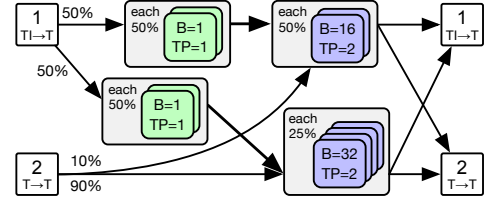


Figure 12. Cornfigurator's deployment plan for InternVL 3 38B ($E \rightarrow L$) under 50% image workload on 16 GPUs.

subplan serves all heavy video-input requests with a disaggregated video encoder, while the other subplan serves the rest of the requests with a monolithic configuration.

6.3 Multimodal Input, Text Output

Setup. MLLMs are a special case of A2A models with multimodal input and text-only output. We evaluate on two MLLMs, InternVL 3 38B [46] and Qwen 3 VL 32B [27], under the ServeGen [36] workload, which includes request timestamps. Half of the requests have image input by default, and we vary this portion in controlled experiments (§6.6).

Baselines. We compare Cornfigurator plans with vLLM [18] and EPD [30], which are restricted versions of Cornfigurator with fixed graph-level configurations (monolithic for vLLM, encoder-prefill-decode disaggregation for EPD) and follow the full Cornfigurator planning pipeline for executor-level configurations and GPU allocation, and ModServe [25] (TP=4, equal encoder/LLM GPU split).⁷

Goodput comparison. Figure 11 compares goodput on 8 and 16 GPUs. For InternVL 3 (Figure 11a), Cornfigurator achieves $1.12 \times - 6.06 \times$ higher goodput compared to baselines on 8 GPUs and $1.27 \times - 3.95 \times$ on 16 GPUs. Figure 12 illustrates the 16 GPU physical deployment plan: the planner disaggregates the large 6B image encoder onto dedicated GPUs. For Qwen 3 VL (Figure 11b), Cornfigurator matches or exceeds all baselines ($1.00 \times - 6.32 \times$), correctly identifying that disaggregation does not help for this model's small encoder. EPD performs poorly because its advantages generally show with short output lengths [30].

⁷vLLM-Omni is built for multimodal *output* models, so it is not included.

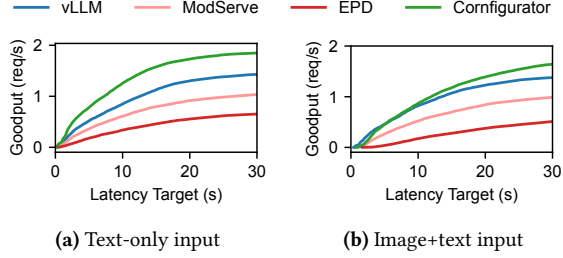


Figure 13. Goodput of requests in InternVL 3 with (a) text-only input and (b) image+text input vs. latency targets under 50% image workload on 16 GPUs.

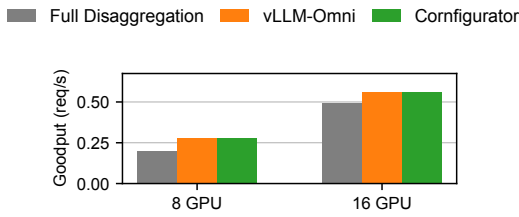


Figure 14. Maximum achievable goodput of Qwen-Image under the DiffusionDB workload on 8 and 16 GPUs.

Per-type goodput vs. latency target. Figure 13 shows that Cornfigurator achieves higher goodput than baselines for both InternVL 3 request types across most latency targets.

6.4 Text Input, Multimodal Output

Setup and baseline. We evaluate Qwen-Image [34] using DiffusionDB [33] prompts with a mixture of low (512×512) and high (1024×1024) resolution requests [20], submitted with Poisson intervals. Cornfigurator’s plan is compared against Full Disaggregation and vLLM-Omni [3]’s plan.

Goodput comparison. Figure 14 compares goodput of Qwen-Image on 8 and 16 GPUs. Qwen-Image is a simple two-component model, and the first component (LLM prefill) is computationally lightweight compared to the second component (DiT). As such, disaggregating the two components does not yield benefits as the first component underutilizes its dedicated GPUs, so Cornfigurator correctly identifies and uses a monolithic deployment plan. Appendix C shows the goodput vs. latency target curve for this model.

6.5 Planner Accuracy

We evaluate the accuracy of the goodput predictions of the plan evaluation pipeline (§4.3), namely the cheap flow-based throughput estimation + Monte Carlo latency estimation (*F&M*) and the more expensive request-level simulator (*Simulator*), by comparing predictions with real serving measurements. Table 3 reports prediction accuracies across Qwen 3 Omni experiments on 8 and 16 GPUs. *F&M* is sufficient for pruning based on aggregate goodput because it is accurate when it comes to aggregate goodput across all request

Input	Output	Mean Abs. Err. (%)		Median Abs. Err. (%)	
		F&M	Simulator	F&M	Simulator
T+I	T	13.0	5.8	8.5	4.9
T+I+V	T	17.5	5.4	17.2	4.0
T+I+A	T	12.9	6.5	11.5	4.4
T+I+V+A	T	16.9	9.3	11.6	7.2
T+I	A	9.6	5.7	3.1	4.6
T+I+V	A	12.0	5.7	5.9	3.2
T+I+A	A	15.6	5.1	14.5	3.4
T+I+V+A	A	12.2	4.9	6.8	4.0
Aggregated		10.7	4.1	8.6	3.3

Table 3. Per-type and aggregate goodput prediction error (%) for Qwen 3 Omni on 8 and 16 GPUs. F&M is flow-based throughput estimation & Monte Carlo latency estimation phases combined, and Simulator is request-level simulation.

Image prob.	vLLM	ModServe	EPD	Cornfigurator
25%	2.49	1.92	0.89	3.60
50%	1.95	1.33	0.63	2.48
75%	1.75	1.71	0.35	1.75

Table 4. InternVL 3 goodput (req/s) under ServeGen with varying image probability on 16 GPUs.

types (mean absolute error 10.7%, median 8.6%). However, it has overall higher error for per-request-type goodput as it does not model queuing effects and inter-type contention at shared executors. On the other hand, *Simulator* is not only more accurate in aggregate goodput (mean absolute error 4.1%, median 3.3%), but also significantly more accurate for individual request types, making it essential for refining the evaluation of plans that are closer to the final plan.

6.6 Workload Drift

Cornfigurator helps runtimes to adapt to workload drift (§4.4). To demonstrate this, we vary the image input probability of the ServeGen workload for InternVL 3 from 25% to 75% on 16 GPUs, and let Cornfigurator re-plan for each workload. Table 4 shows that Cornfigurator consistently matches or outperforms baselines across all image probabilities by adapting to the changing workload. As expected, the planner does not require any re-profiling to adapt to ratio changes, so re-planning takes single- to at most double-digit seconds. Appendix D provides visualizations of Cornfigurator’s plans at different image probabilities.

6.7 Sensitivity Analysis

Cornfigurator’s planner has several parameters as part of its design. In this section, we vary these parameters to understand their effect on goodput and the planning process.

Throughput headroom. After flow-based throughput estimation, the planner scales down the ideal maximum throughput R_d by a headroom factor α (§4.3). Table 5 shows that the

α	Goodput (req/s)	Prediction error
0.5	1.08	-0.6%
0.6	1.59	-1.8%
0.7	1.83	+0.8%
0.8	1.89	-1.6%

Table 5. Effect of throughput headroom α on goodput and planner accuracy. α scales down the rate-matched maximum throughput R_d to $\alpha \cdot R_d$ before latency estimation (§4).

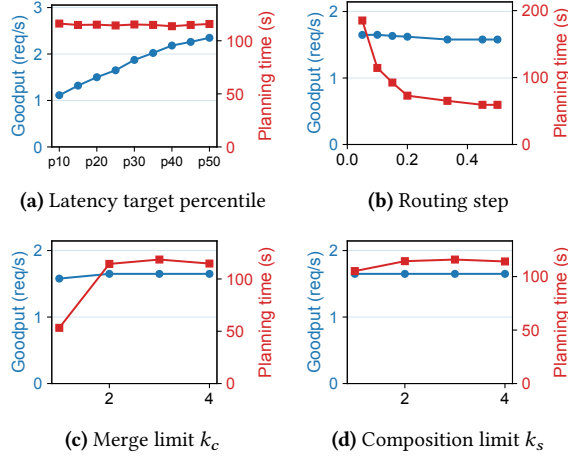


Figure 15. Sensitivity analysis with Qwen 3 Omni on 16 GPUs. Each plot shows planner goodput (left y-axis) and planning time (right y-axis) as one parameter is varied.

planner’s prediction error remains within $\pm 2\%$ across all tested values of α , demonstrating robustness.

Latency target percentile. Figure 15a shows goodput as the latency target percentile is swept from p10 (tight) to p50 (relaxed). As the target loosens, the planner can find higher throughput plans with weaker latency constraints, increasing goodput without increasing planning time. Our default p25 (§6.1) represents a practical middle ground.

Routing probability discretization. When annotating the logical plan with routing probabilities, the planner discretizes probabilities into fixed steps (§4.3). A finer step size would unlock higher goodput plans but also increases planning time, as shown by Figure 15b. We observe that a step size of 0.1 (10% increments) is sufficient to capture goodput benefits without increasing planning time excessively.

Subplan merge and composition limits. During logical plan enumeration, up to k_c simple subplans are merged into compound subplans and up to k_s subplans into a logical plan (§4.2). Figures 15c and 15d confirm that our default $k_c = k_s = 2$ captures most of the benefits of composition with manageable planning overhead.

Phase	Time (s)	Survivors	Fraction
Plan enumeration	77.25	483.41M	100%
Network flow for throughput	3.48	1.95M	0.40%
Monte Carlo for latency	34.23	25	<0.01%
Request-level simulation	0.83	5	<0.01%

Table 6. Planning time breakdown (Qwen 3 Omni, 16 GPUs). Each phase prunes candidates before moving on to the next.

6.8 Cornfigurator Overhead

Profiling and planning are one-time costs per (model, workload) pair, amortized over the deployment lifetime.

Profiling. Profiling averages 4.2 hours on a single node per (model, workload) pair across our evaluation suite. Qwen-Image is the cheapest to profile (27 configurations, 2.6 node-hours) due to its simpler two-component architecture, while Qwen 3 Omni is the most expensive (128 configurations, 9.9 node-hours) due to the number of components and types.

Planning. Table 6 shows how long each phase of the planning pipeline takes for Qwen 3 Omni on 16 GPUs, and how many candidates survive after each phase. This is the most complex planning case in our evaluation suite, and the planner produces a final set of 5 plans from an initial candidate space of nearly 500 million. Without pruning, running request-level simulation on all candidates would take over 4,400 hours; the coarse-to-fine pipeline reduces this to under 2 minutes of total planning time, with only 25 plans (<0.01%) reaching simulation.

7 Related Work

Serving generic and special cases of A2A models. vLLM-Omni [3] and SGLang-Omni [2] are serving runtimes that support generic A2A models, but they do not provide an automated planner. Cornfigurator fills this gap by automatically searching over the space of colocation, disaggregation, and configuration decisions. On the other hand, a large body of work targets special cases of A2A models. For LLM and MLLM serving, many provide serving engines, disaggregation mechanisms, and/or deployment planners [6, 10, 18, 19, 21, 23–25, 30, 39, 42, 44, 45, 47], but do not generalize to generic A2A models. For multimodal generation, existing works on serving optimize inference within a single engine for a single component [4, 11, 20]. Each of these promote a specific deployment strategy for its target model family. Cornfigurator instead treats these strategies as points in a search space and selects the best combination for the given model and workload.

Resource allocation and parallelization for training. Cornfigurator’s deployment planning problem is analogous to large model parallelization for training. Early systems [13, 22, 29] relied on manual, expert-designed strategies, akin to monolithic or fixed disaggregated strategies in serving. Automated planners emerged to search over parallelization

strategies [15–17, 31, 32, 40, 43], analogous to Cornfigurator’s planner that automatically explores the space of deployment strategies rather than unilaterally prescribing one.

8 Conclusion

We present Cornfigurator, the first automated deployment planner for generic Any-to-Any model inference serving. Cornfigurator surfaces the characteristics and requirements of each request type, instead of lumping them together into a single, aggregate objective and constraint. To do so, Cornfigurator systematically explores the vast space of deployment configurations and uses coarse-to-fine statistical evaluation to select the final plan. As more models aim to align with the multimodal nature of human perception and interaction, we believe Cornfigurator’s automated planner will facilitate efficient model development and deployment at scale.

References

- [1] Cornserve. <https://github.com/cornserve-ai/cornserve>.
- [2] SGLang-Omni. <https://github.com/sgl-project/sglang-omni>.
- [3] vLLM-Omni. <https://github.com/vllm-project/vllm-omni>.
- [4] Shubham Agarwal, Subrata Mitra, Sarthak Chakraborty, Srikrishna Karanam, Koyel Mukherjee, and Shiv Kumar Saini. Approximate caching for efficiently serving text-to-image diffusion models. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation*, NSDI’24, USA, 2024. USENIX Association.
- [5] Amey Agrawal, Nitin Kedia, Anmol Agarwal, Jayashree Mohan, Nipun Kwatra, Souvik Kundu, Ramachandran Ramjee, and Alexey Tumanov. On evaluating performance of LLM inference serving systems. *arXiv preprint arXiv:2507.09019*, 2025.
- [6] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *OSDI*, 2024.
- [7] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- [8] Xiaokang Chen, Zhiyu Wu, Xingchao Liu, Zizheng Pan, Wen Liu, Zhenda Xie, Xingkai Yu, and Chong Ruan. Janus-pro: Unified multimodal understanding and generation with data and model scaling. *arXiv preprint arXiv:2501.17811*, 2025.
- [9] Jae-Won Chung, Jeff J. Ma, Jisang Ahn, Yizhuo Liang, Akshay Jajoo, Myungjin Lee, and Mosharaf Chowdhury. Cornserve: A distributed serving system for any-to-any multimodal models. In *ACM CAIS*, 2026.
- [10] Kuntai Du, Bowen Wang, Chen Zhang, Yiming Cheng, Qing Lan, Hejian Sang, Yihua Cheng, Jiayi Yao, Xiaoxuan Liu, Yifan Qiao, Ion Stoica, and Junchen Jiang. PrefillOnly: An inference engine for prefill-only workloads in large language model applications. In *SOSP*, 2025.
- [11] Jiarui Fang, Jinzhe Pan, Xibo Sun, Aoyu Li, and Jiannan Wang. xDiT: an inference engine for diffusion transformers (DiTs) with massive parallelism. *arXiv preprint arXiv:2411.01738*, 2024.
- [12] Yoav HaCohen, Benny Brazowski, Nisan Chiprut, Yaki Bitterman, Andrew Kvochko, Avishai Berkowitz, Daniel Shalem, Daphna Lifschitz, Dudu Moshe, Eitan Porat, Eitan Richardson, Guy Shiran, Itay Chachy, Jonathan Chetboun, Michael Finkelson, Michael Kupchick, Nir Zabari, Nitzan Guetta, Noa Kotler, Ofir Bibi, Ori Gordon, Poriya Panet, Roi Benita, Shahar Armon, Victor Kulikov, Yaron Inger, Yonatan Shifan, Zeev Melumian, and Zeev Farbman. LTX-2: Efficient joint audio-visual foundation model. *arXiv preprint arXiv:2601.03233*, 2026.
- [13] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *NeurIPS*, 2019.
- [14] Hugging Face. Any-to-any models on hugging face. https://huggingface.co/models?pipeline_tag=any-to-any, 2025. Accessed: 2025-06-27.
- [15] Insu Jang, Runyu Lu, Nikhil Bansal, Ang Chen, and Mosharaf Chowdhury. Efficient distributed mllm training with cornstarch. *arXiv preprint arXiv:2503.11367*, 2025.
- [16] Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. Ooblock: Resilient distributed training of large models using pipeline templates. In *SOSP*, 2023.
- [17] Chenyu Jiang, Zhenkun Cai, Ye Tian, Zhen Jia, Yida Wang, and Chuan Wu. DCP: Addressing input dynamism in long-context training via dynamic context parallelism. In *SOSP*, 2025.
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.
- [19] Jiachen Liu, Jae-Won Chung, Zhiyu Wu, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. Andes: Defining and enhancing Quality-of-Experience in LLM-Based text streaming services. *arXiv preprint arXiv:2404.16283*, 2024.
- [20] Runyu Lu, Shiqi He, Wenxuan Tan, Shenggui Li, Ruofan Wu, Jeff J. Ma, Ang Chen, and Mosharaf Chowdhury. TetriServe: Efficiently serving mixed DiT workloads. In *ASPLOS*, 2026.
- [21] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. Helix: Serving large language models over heterogeneous gpus and network via max-flow. In *ASPLOS*, 2025.
- [22] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *SC*, 2021.
- [23] NVIDIA. Nvidia dynamo. <https://github.com/ai-dynamo/dynamo>, 2025.
- [24] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *ISCA*, 2024.
- [25] Haoran Qiu, Anish Biswas, Zihan Zhao, Jayashree Mohan, Alind Khare, Esha Choukse, Íñigo Goiri, Zeyu Zhang, Haiying Shen, Chetan Bansal, Ram Ramjee, and Rodrigo Fonseca. ModServe: Modality- and stage-aware resource disaggregation for scalable multimodal model serving. In *SoCC*, 2026.
- [26] Qwen Team. Qwen-Image 2.0. <https://qwen.ai/blog?id=qwen-image-2.0>, 2025.
- [27] Qwen Team. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [28] Qwen Team. Qwen3.5-Omni: Scaling up, toward native omni-modal agi. <https://qwen.ai/blog?id=qwen3.5-omni>, 2026.
- [29] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506, 2020.
- [30] Gursimran Singh, Xinglu Wang, Yifan Hu, Timothy Tin Long Yu, Linzi Xing, Wei Jiang, Zhefeng Wang, Bai Xiaolong, Yi Li, Ying Xiong, Yong Zhang, and Zhenan Fan. Efficiently serving large multimodal models using EPD disaggregation. In *ICML*, 2025.

- [31] Foteini Strati, Zhendong Zhang, George Manos, Ixeia Sánchez Périz, Qinghao Hu, Tiancheng Chen, Berk Buzcu, Song Han, Pamela Delgado, and Ana Klimovic. Sailor: Automating distributed training over dynamic, heterogeneous, and geo-distributed clusters. In *SOSP*, 2025.
- [32] Zheng Wang, Anna Cai, Xinfeng Xie, Zaifeng Pan, Yue Guan, Weiwei Chu, Jie Wang, Shikai Li, Jianyu Huang, Chris Cai, Yuchen Hao, and Yufei Ding. WLB-LLM: Workload-balanced 4d parallelism for large language model training. In *OSDI*, 2025.
- [33] Zijie J Wang, Evan Montoya, David Munechika, Haoyang Yang, Benjamin Hoover, and Duen Horng Chau. Diffusiondb: A large-scale prompt gallery dataset for text-to-image generative models. In *ACL*, 2023.
- [34] Chenfei Wu, Jiahao Li, Jingren Zhou, Junyang Lin, Kaiyuan Gao, Kun Yan, Sheng ming Yin, Shuai Bai, Xiao Xu, Yilei Chen, Yuxiang Chen, Zecheng Tang, Zekai Zhang, Zhengyi Wang, An Yang, Bowen Yu, Chen Cheng, Dayiheng Liu, Deqing Li, Hang Zhang, Hao Meng, Hu Wei, Jingyuan Ni, Kai Chen, Kuan Cao, Liang Peng, Lin Qu, Minggang Wu, Peng Wang, Shuting Yu, Tingkun Wen, Wensen Feng, Xiaoxiao Xu, Yi Wang, Yichang Zhang, Yongqiang Zhu, Yujia Wu, Yuxuan Cai, and Zenan Liu. Qwen-Image technical report. *arXiv preprint arXiv:2508.02324*, 2025.
- [35] Chengyue Wu, Xiaokang Chen, Zhiyu Wu, Yiyang Ma, Xingchao Liu, Zizheng Pan, Wen Liu, Zhenda Xie, Xingkai Yu, Chong Ruan, et al. Janus: Decoupling visual encoding for unified multimodal understanding and generation. *arXiv preprint arXiv:2410.13848*, 2024.
- [36] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. ServeGen: Workload characterization and generation of large language model serving in production. In *NSDI*, 2026.
- [37] Jin Xu, Zhifang Guo, Jinzheng He, Hangrui Hu, Ting He, Shuai Bai, Keqin Chen, Jialin Wang, Yang Fan, Kai Dang, Bin Zhang, Xiong Wang, Yunfei Chu, and Junyang Lin. Qwen2.5-Omni technical report. *arXiv preprint arXiv:2503.20215*, 2025.
- [38] Jin Xu, Zhifang Guo, Hangrui Hu, Yunfei Chu, Xiong Wang, Jinzheng He, Yuxuan Wang, Xian Shi, Ting He, Xinfu Zhu, Yuanjun Lv, Yongqi Wang, Dake Guo, He Wang, Linhan Ma, Pei Zhang, Xinyu Zhang, Hongkun Hao, Zishan Guo, Baosong Yang, Bin Zhang, Ziyang Ma, Xipin Wei, Shuai Bai, Keqin Chen, Xuejing Liu, Peng Wang, Mingkun Yang, Dayiheng Liu, Xingzhang Ren, Bo Zheng, Rui Men, Fan Zhou, Bowen Yu, Jianxin Yang, Le Yu, Jingren Zhou, and Junyang Lin. Qwen3-Omni technical report. *arXiv preprint arXiv:2509.17765*, 2025.
- [39] Tianhao Xu, Yiming Liu, Xianglong Lu, Yijia Zhao, Xuting Zhou, Aichen Feng, Yiyi Chen, Yi Shen, Qin Zhou, Xumeng Chen, Ilya Shenyuk, Haorui Li, Rishi Thakkar, Ben Hamm, Yuanzhe Li, Xue Huang, Wenpeng Wu, Anish Shanbhag, Harry Kim, Chuan Chen, and Junjie Lai. AIConfigurator: Lightning-fast configuration optimization for multi-framework llm serving. *arXiv preprint arXiv:2601.06288*, 2026.
- [40] Yuanzhong Xu, HyoukJoong Lee, Dehao Chen, Blake Hechtman, Yanping Huang, Rahul Joshi, Maxim Krikun, Dmitry Lepikhin, Andy Ly, Marcello Maggioni, et al. GSPMD: general and scalable parallelization for ML computation graphs. *arXiv preprint arXiv:2105.04663*, 2021.
- [41] Z.AI Team. GLM-Image. <https://z.ai/blog/glm-image>, 2025.
- [42] Chen Zhang, Kuntai Du, Shu Liu, Woosuk Kwon, Xiangxi Mo, Yufeng Wang, Xiaoxuan Liu, Kaichao You, Zhuohan Li, Mingsheng Long, Jidong Zhai, Joseph Gonzalez, and Ion Stoica. Jenga: Effective memory management for serving LLM with heterogeneity. In *SOSP*, 2025.
- [43] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. Alpa: Automating inter- and Intra-Operator parallelism for distributed deep learning. In *USENIX OSDI*, 2022.
- [44] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2023.
- [45] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *OSDI*, 2024.
- [46] Jinguo Zhu, Weiyun Wang, Zhe Chen, Zhaoyang Liu, Shenglong Ye, Lixin Gu, Hao Tian, Yuchen Duan, Weijie Su, Jie Shao, Zhangwei Gao, Erfei Cui, Xuehui Wang, Yue Cao, Yangzhou Liu, Xingguang Wei, Hongjie Zhang, Haomin Wang, Weiye Xu, Hao Li, Jiahao Wang, Nianchen Deng, Songze Li, Yanan He, Tan Jiang, Jiapeng Luo, Yi Wang, Conghui He, Botian Shi, Xingcheng Zhang, Wenqi Shao, Junjun He, Yingting Xiong, Wenwen Qu, Peng Sun, Penglong Jiao, Han Lv, Lijun Wu, Kaipeng Zhang, Huipeng Deng, Jiaye Ge, Kai Chen, Limin Wang, Min Dou, Lewei Lu, Xizhou Zhu, Tong Lu, Dahua Lin, Yu Qiao, Jifeng Dai, and Wenhai Wang. InternVL3: Exploring advanced training and test-time recipes for open-source multimodal models. *arXiv preprint arXiv:2504.10479*, 2025.
- [47] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Tian Tang, Qinyu Xu, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. NanoFlow: Towards optimal large language model serving throughput. In *OSDI*, 2025.

A Compute-Proportional Deadlines

For each request type t , let ℓ_t denote the sum of per-component compute latencies along the computation path of type t . This quantity is determined by the model architecture and hardware, independent of the deployment plan. Any request's latency can be written as $\ell_r = \ell_t \cdot X_t$, where $X_t = \ell_r / \ell_t$ captures queuing delay, load-dependent effects, and distributional shape. Per-type deadlines are set as $L_t = L \cdot \ell_t / \ell_{\max}$, where $\ell_{\max} = \max_t \ell_t$.

Theorem A.1. *Under per-type deadlines $L_t = L \cdot \ell_t / \ell_{\max}$, the per-type CDF at the deadline threshold is*

$$\Pr(\ell_r \leq L_t \mid t_r = t) = F_{X_t}(L/\ell_{\max}), \quad (2)$$

where F_{X_t} is the CDF of X_t . That is, all types evaluate their shape CDF at the same argument $L/\ell_{\max}$, independent of their scale ℓ_t .

Proof. Substituting $\ell_r = \ell_t \cdot X_t$ and $L_t = L \cdot \ell_t / \ell_{\max}$:

$$\begin{aligned} \Pr(\ell_r \leq L_t \mid t_r = t) &= \Pr\left(\ell_t \cdot X_t \leq L \cdot \frac{\ell_t}{\ell_{\max}}\right) \\ &= \Pr(X_t \leq L/\ell_{\max}) \\ &= F_{X_t}(L/\ell_{\max}). \end{aligned} \quad (3)$$

The scale ℓ_t cancels, leaving only the shape distribution X_t . $\square$

Without per-type deadlines (i.e., a single global threshold L for all types), the per-type CDF is $F_{X_t}(L/\ell_t)$, where the argument L/ℓ_t varies across types by an order of magnitude (e.g., $L/4.5$ vs. $L/18$ for text and audio output). Per-type deadlines eliminate this scale dependence entirely.

Corollary A.2. *If all request types share the same distributional shape ($X_t \stackrel{d}{=} X$ for all t), then all per-type CDFs evaluated at their respective deadlines are identical, and the percentile constraint binds equally on every request type.*

B Qwen 3 Omni Per-Type Goodput vs. Latency Target

Figure 16 shows the goodput vs. latency target curves for all 8 request types (Section 6.2 Figure 9 highlights TIVA→T and TIVA→A). Cornfigurator achieves higher goodput compared to both baselines across all request types and latency targets. The gap is especially pronounced for text-output types (top four), where Cornfigurator's colocation decisions reduce encoder interference and improve LLM throughput. Audio-output types (bottom four) exhibit much longer latencies than text-output types due to the talker's iterative generation, and Cornfigurator's advantage comes from better allocation of talker-vocoder replicas.

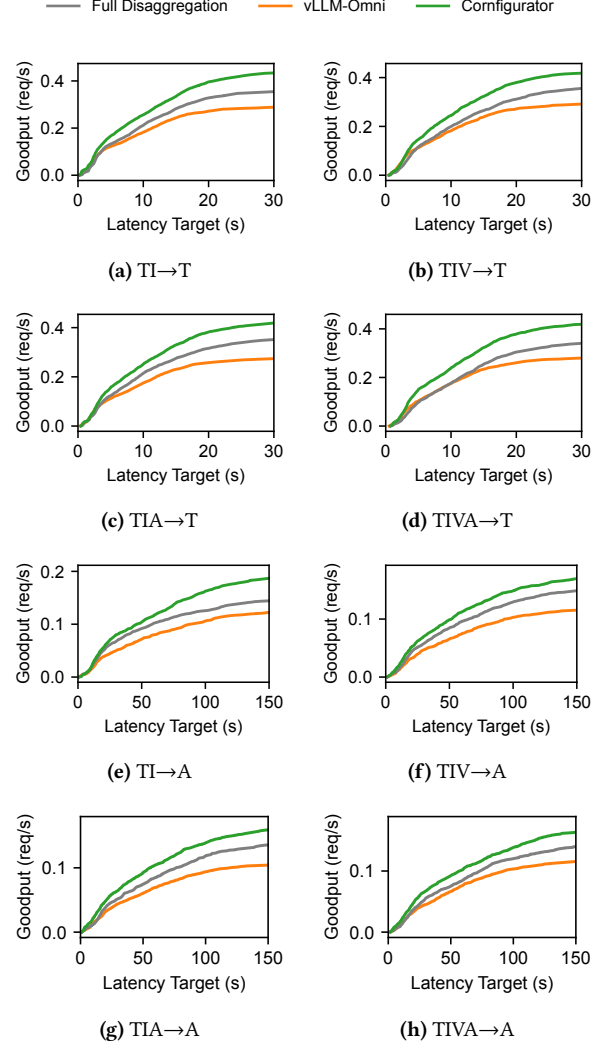


Figure 16. Goodput of all 8 request types in Qwen 3 Omni vs. latency targets under 1/3 audio workload on 16 GPUs. (a)–(d): text output. (e)–(h): audio output.

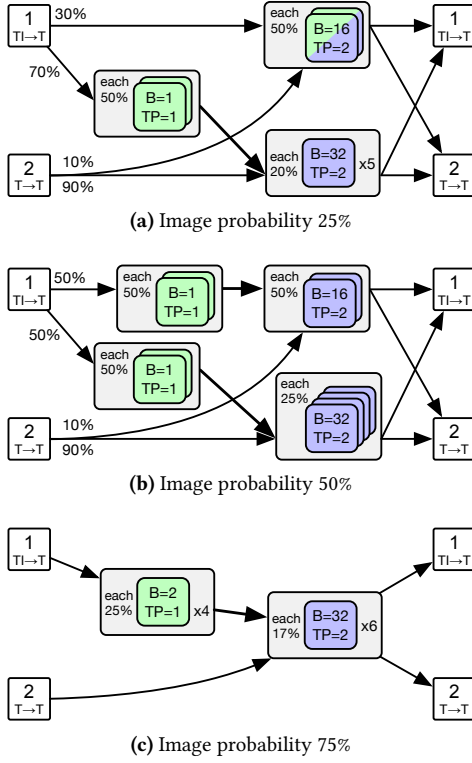


Figure 18. Cornfigurator’s deployment plans for InternVL 3 38B (E→L) on 16 GPUs under workload drift in terms of image request type fraction.

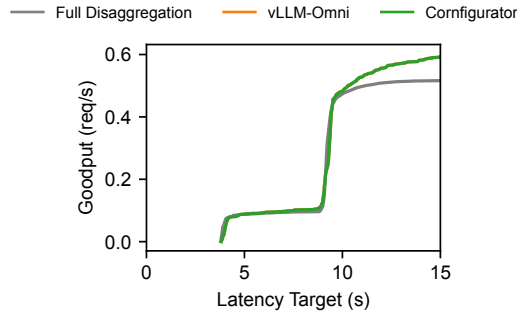


Figure 17. Goodput of requests in Qwen-Image with text input and image output vs. latency targets on 16 GPUs.

C Qwen-Image Per-Type Goodput vs. Latency Target

Figure 17 shows the goodput vs. latency target curve for Qwen-Image’s single request type (text input, image output). Cornfigurator overlaps with vLLM-Omni because the planner correctly identified vLLM-Omni’s monolithic plan as optimal. Full Disaggregation uses 2 LLM replicas + 7 SP=2 DiT replicas, as uniform sequence parallelism is enforced in this restricted search space. The workload contains requests at 512×512 and 1024×1024 resolutions (weighted toward 1024), which have drastically different compute requirements. The two visible inflection points in the curve correspond to these resolutions: the first bump around the 512×512 compute time and the second around 1024×1024 . At relaxed latency targets, the curves diverge because the monolithic deployment dedicates more GPUs to DiT replicas (no encoder GPU waste), achieving higher throughput.

D InternVL Workload Drift

Figure 18 shows Cornfigurator’s deployment plans for InternVL 3 38B on 16 GPUs under workload drift in terms of the fraction of image requests. As image probability grows from 25% to 75%, the incoming workload is essentially being more image-heavy and exerts greater pressure on the image encoder. This is why, when image probability increase from 25% to 50%, Cornfigurator’s planner decides to additionally disaggregate the image encoder from the original monolithic executor. As image probability further increases to 75%, the image encoder becomes a more severe bottleneck of the system, and Cornfigurator responds by increasing the batch size of the image encoder to improve its throughput. At this point, the LLM executor that used to be running with batch size 16 becomes the new bottleneck, so Cornfigurator also increases its batch size to 32—at which point the resulting plan is equivalent to pooling all encoders together, and all LLM executors together, resulting in the final plan.